

■ ЛЕКЦИЯ 1 · ИНСТРУМЕНТЫ РАЗРАБОТКИ



Linux и командная строка

Как устроена ОС, на которой работает вся ML-инфраструктура,
и как уверенно управлять ей из терминала

Чернов Евгений

Магистратура «Искусственный интеллект», НИУ ВШЭ

План занятия

1

Зачем Linux

ОС серверов, облаков и ML-инфраструктуры

2

Ядро и shell

kernel, userspace, оболочка

3

Файловая система

FHS, inode, жёсткие и мягкие ссылки

4

Права и процессы

rwX, sudo, PID, сигналы

5

Потоки и конвейеры

stdin/stdout/stderr, |, редиректы

6

Инструменты CLI

ls, grep, find, sed, awk, apt

Зачем ML-инженеру Linux?

Учитесь вы, скорее всего, на Windows или macOS, но код обучается и работает на сервере. А сервер — почти всегда Linux.

```
● ● ● удалённый сервер

$ ssh gpu-server
$ uname -sr
Linux 5.15.0-91-generic
$ nvidia-smi
Tesla V100 · 32 GB
# весь ML-стек — здесь
```



Серверы и облако — AWS, GCP, кластеры — почти всегда Linux



ML-стек как дома — CUDA, PyTorch, Docker — родная среда



Открытость — бесплатно, прозрачно, полностью скриптуется



Автоматизация — всё управляется командами, а не мышкой

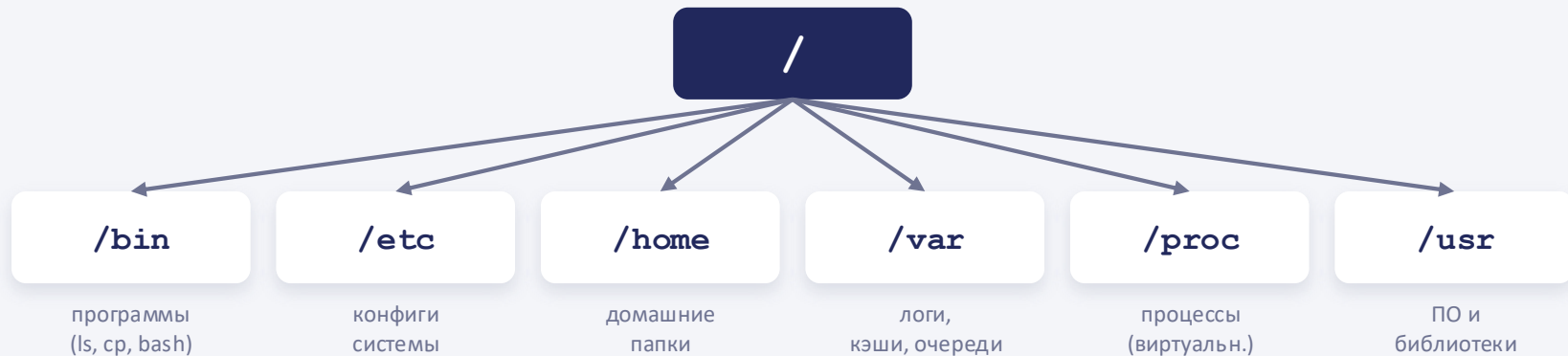
Ядро, userspace и оболочка

Между пользователем и «железом» — несколько слоёв. Команды идут вниз к ядру через системные вызовы, а результат возвращается вверх.



FHS: одно дерево от корня /

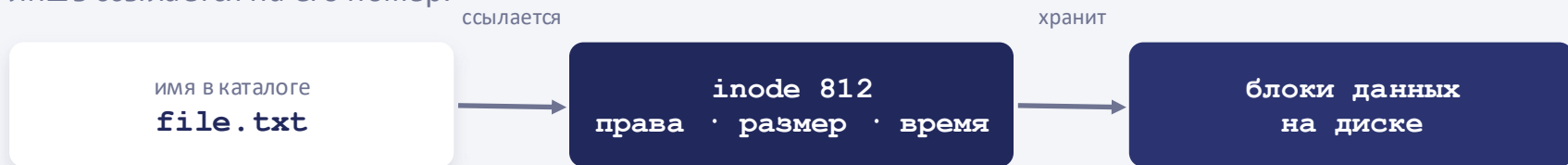
В Linux нет дисков C:/ и D:/ — всё образует единое дерево, растущее из корня «/». Стандарт FHS задаёт, что где лежит.



Устройства монтируются в папки (**флешка** → **/media**, **диск** → **/mnt**) — тоже часть общего дерева.

inode: имя ≠ содержимое

Имя файла и его данные — разные сущности. Содержимое и метаданные хранит inode; имя в каталоге лишь ссылается на его номер.



Жёсткая ссылка

`ln file copy`

- второе имя того же inode
- переживает удаление оригинала

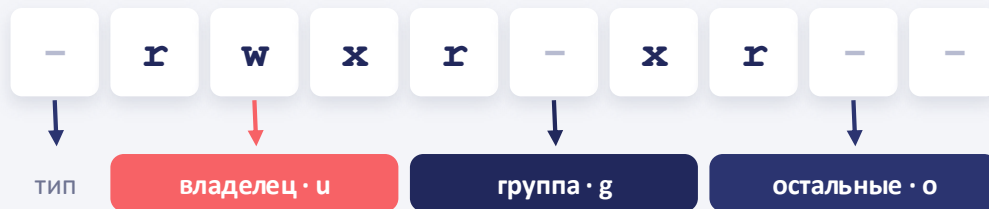
Символьная ссылка (symlink)

`ln -s file link`

- хранит путь; ведёт и на папки
- «повиснет», если цель удалить

rwX: КТО И ЧТО МОЖЕТ ДЕЛАТЬ С ФАЙЛОМ

У файла есть владелец, группа и биты доступа: r (чтение), w (запись), x (исполнение) — отдельно для трёх категорий.



терминал

```
$ ls -l run.sh
-rwxr-xr-- 1 ivan ml run.sh
$ chmod +x run.sh      # +исполнение
$ chmod 640 key        # rw-r-----
$ chown ivan:ml data   # владелец
```

r чтение · 4 **w** запись · 2
x исполнение · 1 → 7 = **rwX**

sudo — выполнить команду от root

Как читать строку ls -l

Команда ls -l печатает по строке на файл. В строке несколько полей: тип и права, число ссылок, владелец, группа, размер, дата и имя.



терминал

```
$ ls -l
-rw-r--r-- 1 ivan ml 4096 model.py
drwxr-xr-x 2 ivan ml 4096 data
$ ls -la # + скрытые файлы
```

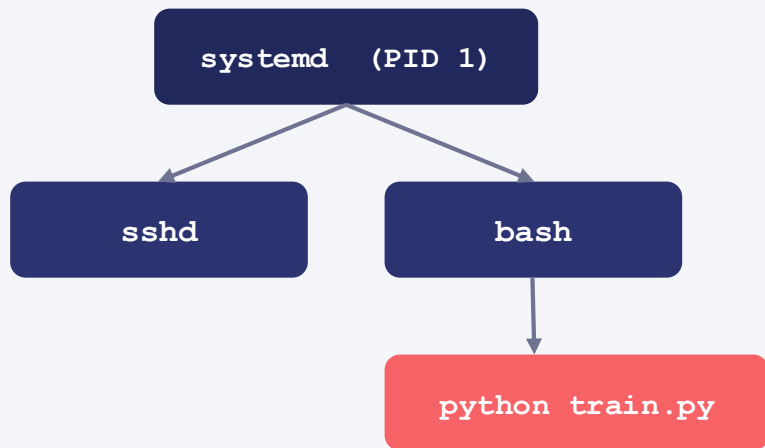
Первый символ — это тип:

– файл d каталог l ссылка.

Размер — в байтах.

PID, дерево процессов и сигналы

Каждая запущенная программа — процесс со своим номером PID. У любого процесса есть родитель, поэтому все они образуют дерево.



родитель → потомок

смотрим и завершаем

```
$ ps aux | grep python
ivan 4821 python train.py
$ kill 4821      # SIGTERM
$ kill -9 4821  # SIGKILL
```

SIGINT (Ctrl+C) прервать процесс

SIGTERM попросить завершиться мягко

SIGKILL (kill -9) убить принудительно, без уборки

Ctrl+Z · **&** пауза · запуск в фоне (jobs/fg/bg)

stdin / stdout / stderr, пайпы и редиректы

У каждого процесса есть три стандартных потока. Их можно перенаправлять в файлы и соединять команды в конвейеры.



редиректы

```
$ ls > files.txt           # stdout в файл
$ echo x >> f.txt          # дозапись
$ cmd 2> err.log           # stderr в файл
$ cmd < in.txt             # stdin из файла
```

Конвейер | — stdout одной команды становится stdin следующей:



Как данные текут через конвейер

Разберём конвейер по шагам. Каждая следующая команда получает на вход результат предыдущей и обрабатывает его дальше.



Каждый | соединяет stdout слева и stdin справа. Промежуточные файлы не создаются — данные текут через память.

01

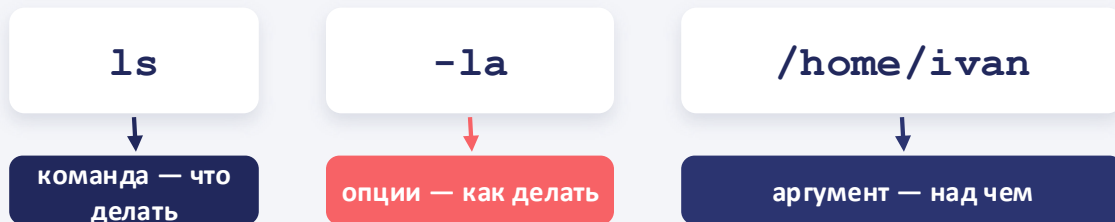
 РАЗДЕЛ

Инструменты командной строки

Небольшие утилиты, из которых по принципу конвейера собираются мощные операции

Анатомия команды: что вы набираете

Почти любая команда состоит из трёх частей: сама команда, опции (флаги) и аргументы. Опции меняют поведение, аргументы говорят, над чем работать.



терминал

```
$ ls                # без опций
$ ls -l ~/project   # опция + аргумент
$ cp -r src/ backup/ # два аргумента
$ grep -n TODO app.py
```

-l — короткая опция; **-la** = -l -a.

--all — длинная форма. **--help** есть почти у всех.

Где я? Пути и перемещение по дереву

Первая команда в незнакомой системе — «где я». Путь бывает абсолютный (от корня /) и относительный (от текущей папки).

терминал

```
$ pwd
/home/ivan
$ cd project/src    # относит. путь
$ cd /etc           # абсолютный путь
$ cd ~              # домой
$ cd ..             # на уровень вверх
$ cd -              # в прошлую папку
```

/ корень всего дерева

~ домашняя папка (/home/вы)

. текущая папка

.. родительская папка

Tab дополнить имя — жмите чаще



Абсолютный путь начинается с /, относительный — от того, где вы сейчас (проверьте **pwd**).

Базовые утилиты, которыми пользуются каждый день

ls

список файлов (-l подробно, -a со скрытыми)

cp

копировать файл или папку (-r рекурсивно)

rm

удалить (-rf — рекурсивно и без вопросов, опасно)

cd

перейти в каталог (cd .. вверх, cd ~ домой)

mv

переместить или переименовать

cat

вывести содержимое файла в stdout

Первая сессия: заводим проект

Соберём всё вместе: создадим папку проекта, положим в неё файл и запустим его — только команды, без мышки.

● ● ● терминал — реальная сессия

```
$ pwd
/home/ivan
$ mkdir project && cd project
$ mkdir src data
$ echo "print('hi')" > src/app.py
$ ls src
app.py
$ cat src/app.py
print('hi')
$ python src/app.py
hi
```

1

pwd

узнать, где мы находимся

2

mkdir · cd

&& — вторую, если первая успешна

3

echo >

записать текст в новый файл

4

ls · cat

проверить, что получилось

5

python

запустить готовый скрипт

Просмотр файлов: cat, less, head, tail

Файл не обязательно открывать редактором. Для быстрого просмотра есть четыре инструмента — под разные задачи.

терминал

```
$ cat app.py           # весь файл сразу
$ less huge.log        # листать, выход q
$ head -n 20 app.log   # первые 20 строк
$ tail -n 20 app.log   # последние 20
$ tail -f app.log      # следить онлайн
```

cat

весь файл в терминал; для коротких

less

постранично; листать ↑↓, выход q

head

начало файла (по умолч. 10 строк)

tail

конец файла; -f следит за новыми



Для логов **tail -f** незаменим: показывает новые строки, пока программа ещё работает.

grep — строки, find — файлы

grep ищет строки по шаблону внутри файлов; find ищет сами файлы по имени, типу, размеру и времени.

 grep — поиск строк

```
$ grep "error" app.log      # найти
$ grep -in "TODO" *.py     # без регистра
$ grep -rc "API" src/       # рекурс., счёт
$ grep -v "DEBUG" app.log   # кроме DEBUG
```

find . -name по имени и маске (*.py)

find -type d только каталоги

find -size +100M по размеру файла

find -mtime -1 по времени изменения



Вместе: **find . -name "*.py" | xargs grep "def train"** — find находит файлы, grep ищет строки внутри.

sed и awk: потоковая обработка

Оба читают stdin построчно и преобразуют его. Правило: sed — про строки и замены, awk — про поля (колонки) и арифметику.

● ● ● sed — редактор потока

```
$ sed 's/foo/bar/g' f # заменить
$ sed 's/ /_/g' names # пробел → _
$ sed -n '2,5p' f # строки 2-5
$ sed '/DEBUG/d' log # убрать
```

● ● ● awk — работа с колонками

```
$ awk '{print $1}' f # колонка 1
$ awk -F: '{print $1}' # поле по :
$ awk '$3>100' data # фильтр
$ awk 'END{print NR}' # число строк
```



\$0 — вся строка, **\$1**, **\$2**, ... — её поля. Так за одну строку считают статистику по логам и таблицам.

Переменные окружения, PATH и .bashrc

Переменные окружения настраивают поведение программ. PATH перечисляет папки, где shell ищет команды.

```
● ● ● терминал

$ echo $HOME
/home/ivan
$ export API_KEY="secret"
$ echo $PATH
/usr/local/bin:/usr/bin:/bin
$ which python
/usr/bin/python
```

PATH

список папок; команда ищется в них по порядку

\$VAR / export

прочитать / передать дочерним процессам

.bashrc

выполняется при старте bash: alias, export, PATH

env

показать все переменные окружения

Менеджеры пакетов: apt

Программы в Linux ставят не с сайтов, а из репозиториев — менеджером пакетов. В Debian и Ubuntu это apt.

● ● ● терминал

```
$ sudo apt update          # список
$ sudo apt install htop    # поставить
$ sudo apt upgrade        # обновить
$ apt search postgres     # найти
$ sudo apt remove htop    # удалить
```

apt update

обновить индекс пакетов (не сами программы)

apt install

поставить пакет вместе с зависимостями

apt remove

удалить установленный пакет

dnf · brew

менеджеры Fedora и macOS — тот же смысл



apt ставит системные пакеты; для библиотек Python — **pip** / **conda** в виртуальном окружении. Не путайте уровни.

Горячие клавиши и как получить помощь

Несколько приёмов ускоряют работу в разы — и спасают, когда кажется, что «терминал завис».

Tab

дополнить имя файла или команды

↑ / ↓

листать историю прошлых команд

Ctrl+R

поиск по истории команд

Ctrl+C

прервать текущую команду

Ctrl+D

конец ввода (закрыть stdin)

Ctrl+L

очистить экран (= clear)

q

выйти из less / man

man · --help

справка по команде и опциям

Ошибки новичка и что с ними делать

Эти сообщения вы увидите в первый же день. Они не страшные — каждое подсказывает, что именно пошло не так.

`command not found`

опечатка или пакет не установлен — проверьте which

`No such file...`

не тот путь: сделайте pwd и ls, дополняйте по Tab

`Permission denied`

нет прав: посмотрите ls -l, при нужде chmod / sudo

`rm -rf` — без корзины

удаление окончательно; всегда перечитайте путь

`>` вместо `>>`

`>` перезаписывает файл целиком, `>>` дописывает в конец

«терминал завис»

команда ждёт ввод: нажмите Ctrl+C или Ctrl+D

Что важно унести

1

Linux — среда ML

серверы, GPU, Docker — весь стек живёт здесь

2

Всё — файлы и процессы

дерево от /, права rwx, процессы с PID

3

Потоки и конвейеры

stdin/stdout/stderr, | и редиректы
соединяют команды

4

Терминал — это сила

ls/grep/find/sed/awk/apt автоматизируют
работу

Дальше — **Git: как версионировать код**. Практика — тест по Linux.